Ciudad interactiva con OpenGL



NOMBRE: JOSUE DAVID TORREZ DURAN

REGISTRO: 222010045

DOCUMENTACIÓN TÉCNICA
CIUDAD CONDUCCIÓN

Análisis del proyecto Java/OpenGL contenido en **CiudadConduccion(1).zip**. Se excluye deliberadamente la funcionalidad de la carreta, sus métodos, variables y comportamiento.

1. Objetivo y funcionamiento general

El programa representa una ciudad tridimensional por la que el usuario conduce un automóvil rojo. Permite acelerar, frenar, girar, cambiar de cámara, recorrer calles con colisiones, visualizar un ciclo de iluminación manual, observar tráfico autónomo y completar entregas. La interfaz muestra información del estado, avisos de sector y un minimapa.

2. Tecnologías y ejecución

Se utiliza **Java 17**, **Maven**, **LWJGL 3.3.3**, GLFW para ventana y entrada, y OpenGL con shaders para dibujar e iluminar. En IntelliJ IDEA, abrir la carpeta que contiene **pom.xml**, cargar Maven y ejecutar el método **main** de **com.graphics.clase4**. La aplicación necesita un entorno compatible con OpenGL.

3. Organización del código

clase1.java: Motor gráfico base: ventana, bucle, mapa, geometría, cámaras básicas, shaders y dibujo de la escena.

clase2.java: Automóvil del jugador: posición, velocidad, dirección, entrada de teclado, cámaras y colisiones. En esta documentación se omite la carreta.

clase3.java: Iluminación: ambiente de día/noche, farolas y faros; configuración de luces y shader.

clase4.java: Juego completo: entregas, decoraciones, semáforos, tráfico NPC, minimapa, avisos y panel.

InterfazCiudad.java: Genera textos mediante una imagen Java AWT y los representa como textura en OpenGL.

ConduccionTest.java y FuncionesTest.java: Pruebas automáticas de distribución, colisiones, controles, entregas y tráfico.

4. Herencia y flujo de ejecución

La jerarquía es **clase4** → **clase3** → **clase2** → **clase1**. Cada nivel amplía al anterior. El punto de entrada inicia el motor, crea ventana y recursos gráficos, procesa eventos y repite el ciclo principal: leer teclas, calcular el tiempo transcurrido, actualizar vehículos y entregas, configurar la cámara y luces, dibujar escena y superposiciones, y presentar el fotograma. Al salir libera recursos.

5. Mapa, geometría y colisiones

El mapa es una matriz **MAPA de 11 × 11**, con celdas de 10 unidades, que distingue calles, edificios y parques. La clase base transforma coordenadas de cuadrícula a posiciones 3D y dibuja formas mediante cubos escalados o rotados. Las comprobaciones de circulación impiden atravesar manzanas y límites; la versión final también considera vehículos.

6. Controles del automóvil

Tecla	Acción
W / ↑	Acelerar hacia delante
S / ↓	Retroceder
A / ← y D / →	Girar a izquierda y derecha
Espacio	Frenado fuerte
C	Cambiar modo de cámara
Q / E	Rotar vista en cámara panorámica
N	Alternar día y noche
F	Encender/apagar faros
H	Mostrar/ocultar ayuda y panel
M	Alternar minimapa
R	Reiniciar partida y tráfico
Esc	Cerrar aplicación

7. Movimiento y cámaras

En cada actualización se ajusta la velocidad según aceleración, reversa y resistencia al movimiento; la barra espaciadora aumenta la desaceleración. La orientación determina el desplazamiento en los ejes X y Z. Se limita la velocidad y se valida la posición antes de mover el auto. Las cámaras disponibles incluyen seguimiento en tercera persona, vista aérea y panorámica general; la última admite giro orbital.

8. Iluminación y aspecto visual

El programa inicia de día. La tecla N conmuta manualmente el estado nocturno y F controla los faros. La iluminación combina ambiente, farolas y luces delanteras; el shader calcula efectos locales sin generar sombras proyectadas. La ciudad añade edificios con ventanas, parques, árboles, bancos, fuente animada, pasos peatonales y semáforos.

9. Tráfico autónomo y semáforos

La clase **AutoTrafico** mantiene la ruta, posición, orientación y color de cada NPC. **crearTrafico()** crea cinco vehículos y **rutaSuave()** prepara trayectorias que incluyen calles interiores, suavizando los giros. **actualizarTrafico(dt)** desplaza los autos y comprueba obstáculos delante para evitar choques. En el código analizado, los NPC no se detienen por el semáforo: solo frenan ante obstáculos. Los semáforos tienen un ciclo visual de 12 segundos: 5 rojo, 5 verde y 2 amarillo.

10. Entregas y progreso

El juego define **tres destinos** mediante coordenadas X/Z. Se marca visualmente el destino actual; para completar una entrega, el automóvil debe acercarse y frenar. Al lograrla aumenta el contador y aparece la siguiente. El título de la ventana refleja el progreso y el tiempo; al completar las tres se informa la finalización. Reiniciar restablece posición, progreso y tráfico.

11. Interfaz y minimapa

El minimapa representa las calles y posiciones de jugador, tráfico y destino, con norte hacia arriba. **InterfazCiudad** pinta información textual en una imagen que se sube como textura. La ayuda está oculta inicialmente y se alterna con H. Los avisos de sector ESTE, SUR y SURESTE aparecen superpuestos con transición de entrada y salida, sin carteles físicos.

12. Métodos importantes para modificar el proyecto

clase1.actualizar / dibujarFrame: Base de actualización y renderizado.

clase2.actualizar / puedeCircular: Física del jugador y validación del desplazamiento.

clase2.configurarCamara: Cálculo de vista según modo seleccionado.

clase3.prepararLuces / fragmentShader: Parámetros y comportamiento de iluminación.

clase4.actualizar: Coordina tráfico, progreso y animación.

clase4.crearTrafico / rutaSuave / actualizarTrafico: Rutas y comportamiento de autos NPC.

clase4.dibujarSemaforo / faseSemaforo: Apariencia y ciclo de señales.

clase4.dibujarFrame / dibujarExtras: Composición de escena, minimapa e indicadores.

InterfazCiudad.pintar / dibujar: Construcción y presentación del panel de texto.

13. Pruebas y observaciones

Las clases de test contienen verificaciones de mapa, accesibilidad, colisiones, controles, reinicio, entregas y recorridos del tráfico. El README del ZIP describe un comportamiento anterior de algunos NPC; para esta documentación se priorizó la implementación efectiva de **clase4.java**, que indica expresamente que los vehículos no esperan semáforos. No se ejecutaron pruebas gráficas durante la elaboración de este documento.

14. Conclusión

CiudadConduccion organiza un simulador de conducción por capas: motor gráfico, vehículo, iluminación y mecánicas finales. Esta separación permite localizar rápidamente los cambios: controles y física en clase2, luces en clase3, y tráfico, entregas, señales e interfaz en clase4. El análisis se limita al funcionamiento ajeno a la carreta.